

P1403R0

Experience Report: Implementing a Coroutines TS Frontend to an Existing Tasking Library

Draft Proposal, 2019-01-16

This version:

wg21.link/P1403

Issue Tracking:

[GitHub](https://github.com)

Author:

[David S. Hollman](#)

Audience:

SG1, EWG, WG21

Project:

ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Table of Contents

- 1 Revision History**
 - 1.1 P1403R0
- 2 Background**
- 3 Implementation**
- 4 Benchmarks**
- 5 Appendix: Source code**

§ 1. Revision History

§ 1.1. P1403R0

- Initial version

§ 2. Background

Kokkos is a performance portability library for HPC applications. It provides abstractions for writing algorithms that are generic over the details of their execution and storage, and provides backends for execution with CPUs, GPUs, and other accelerators.

While most of Kokkos is focused on providing loop-level abstractions (like `for_each` and `reduce`), it also provides a facility for fine-grained task DAG execution. The application-level interface for Kokkos tasking takes the form of a typical `async/future` programming model with some important caveats. Most importantly, futures in Kokkos cannot be waited on; instead, the user must respawn the current task with the desired future as a dependence. The respawned task will start over from the beginning when the given dependence is ready, and it is up to the user to store any state needed across respawns and to return to the previous point of progress before the respawn was requested. This is perhaps best illustrated by way of the ubiquitous recursive Fibonacci example:

```

template <class Scheduler>
struct Fib {
    using value_type = long;
    using future_type = Kokkos::BasicFuture<long, Scheduler>;
    using team_member_type = typename Scheduler::member_type;

    const value_type n;
    future_type deps[2];

    KOKKOS_INLINE_FUNCTION // things like __device__, when appropriate
    void operator()(team_member_type const& member, value_type& result) {
        auto sched = member.scheduler();
        if(n < 2) {
            // recursive base case:
            result = n;
        }
        else if(deps[0].is_ready() && deps[1].is_ready()) {
            // this is the respawn case, since the dependences will only
            // be ready after respawn:
            result = deps[0].get() + deps[1].get();
        }
        else {
            // Spawn tasks for Fib(n-1) and Fib(n-2) and store their futures
            // in a member variable
            deps[0] = Kokkos::task_spawn(
                Kokkos::TaskSingle(sched, Kokkos::TaskPriority::High),
                Fib{n - 2}
            );
            deps[1] = Kokkos::task_spawn(
                Kokkos::TaskSingle(sched),
                Fib{n - 1}
            );
            // Aggregate the dependences into one future
            auto fib_all = Kokkos::when_all(deps, 2);
            // Respawn this task dependent on the aggregate future
            Kokkos::respawn(this, fib_all, Kokkos::TaskPriority::High);
        }
    }
};

```

With only a couple of hours of work, we were able to use the Coroutines TS to create a wrapper to this interface that afforded the following code:

```

template <class Scheduler>
struct FibCoroutine {
    using value_type = long;
    using coroutine_scheduler_type = BasicCoroutineScheduler<Scheduler>;

    value_type n;

    typename coroutine_scheduler_type::template coroutine_return_type<long>
    operator()(coroutine_scheduler_type& sched) {
        if(n < 2) {
            co_return n;
        }
        else {
            auto f_2 = sched.spawn(FibCoroutine{n-2}, Kokkos::TaskPriority::High);
            auto f_1 = sched.spawn(FibCoroutine{n-1});
            auto [v1, v2] = co_await sched.when_all(f_1, f_2);
            co_return v1 + v2;
        }
    }
};

```

The implementation of this wrapper required no modification to Kokkos itself, and benchmarks showed zero (or less!) overhead for this interface at runtime. This wrapper code is presented below in its entirety. It has been included in contiguous form (along with driver code to run the benchmarks) in an appendix, in case the section below is difficult to read as is.

The code for the wrapper was prepared by team members with little or no experience using the Coroutines TS. We present this as further evidence that the Coroutines TS is sufficiently baked and should be merged into the C++20 draft.

§ 3. Implementation

The basic implementation strategy employed involved wrapping the Kokkos `Scheduler` abstraction in a coroutine-aware scheduler with similar semantics:

```

template <class Scheduler>
struct BasicCoroutineScheduler {

```

It holds an instance of the Kokkos scheduler so that the coroutine scheduler can delegate to it:

```
Scheduler m_scheduler;
```

We use a Kokkos future to communicate the user's suspension points as dependences in the Kokkos tasking backend:

```
using future_type = Kokkos::BasicFuture<void, Scheduler>;
```

The coroutine scheduler provides `Awaitable` nested types to be returned by `scheduler.spawn()` and `scheduler.when_all()`:

```
template <class CoroutineFunctor>
struct SpawnedAwaitable;
template <class...>
struct WhenAllAwaitable;
```

The coroutine scheduler also provides the return type for the user's coroutine, through which all of the necessary plumbing is communicated to the compiler:

```
template <class T>
struct coroutine_return_type {
    // assume value_type is not void for brevity
    using value_type = T;
    // forward declaration:
    struct promise_type;
    // for brevity:
    using coroutine_handle = std::experimental::coroutine_handle<promise_ty
```

We store the `coroutine_handle` in a data member of the return object:

```
coroutine_handle handle;
```

The coroutine promise type holds the storage for the result as well as the future representing the dependence at the current suspend point:

```
struct promise_type {  
  
    std::optional<value_type> result;  
    future_type* m_current_dep = nullptr;  
};
```

Promise creation must be followed by an initial suspend in order to set up `m_current_dep`:

```
std::experimental::suspend_always initial_suspend() { return { }; }
```

The coroutine return must be followed by a suspend in order to extract the result before the promise is destroyed:

```
std::experimental::suspend_always final_suspend() { return { }; }
```

When the user's coroutine returns, we simply store the result:

```
template <class Value>  
void return_value(Value&& value) {  
    result = std::forward<Value>(value);  
}
```

The returned object holds the coroutine handle, which is constructed from the promise directly:

```
coroutine_return_type<value_type>  
get_return_object() {  
    return { coroutine_handle::from_promise(*this) };  
}
```

Now comes the critical part. When the user `co_await`s on the result of a `spawn` or `when_all`, we need to point the future that communicates the suspension of the current coroutine to the dependence that the suspension needs to wait on. Fortunately, `await_transform` allows us to do that:

```

template <class ValueType>
typename SpawnedAwaitable<ValueType>::template SpawnedPromise<promise
await_transform(SpawnedAwaitable<ValueType>& awaitable) {
    return { awaitable.m_done_future, *this };
}

```

By storing a reference to the parent promise in the awaitable (`*this` in the above code), the transformed awaitable is able to communicate its dependence through to the Kokkos backend when `await_suspend` is called. (We don't do this now because `await_ready` may return true, obviating the need to communicate the dependence to the backend.) A similar thing happens for the `when_all` case:

```

template <class... Awaitables>
typename WhenAllAwaitable<Awaitables...>::template SpawnedPromise<pro
await_transform(WhenAllAwaitable<Awaitables...>const & awaitable) {
    return {
        awaitable.m_done_future,
        awaitable.m_value_futures,
        *this
    };
}

```

And that's it for the `promise_type`. We include a couple of convenience methods in the `coroutine_return_type` to make the rest of the code more readable, and then close out that class also:

```
};

bool is_done() {
    return bool(handle.promise().result);
}

value_type& get_result() {
    return *handle.promise().result;
}

coroutine_handle handle;

};
```

To communicate the dependence structure of the user's coroutine to the backend, we use a Kokkos task functor, just like the one in the old version of the Fibonacci example above:

```
template <class CoroutineFunctor>
struct TaskFunctor {
    using value_type = typename CoroutineFunctor::value_type;
```

We store the user's functor itself, the coroutine return object, and the suspension dependence as members of this `TaskFunctor`, so that we can find them when Kokkos respawns us:

```
CoroutineFunctor m_functor;
std::optional<coroutine_return_type<value_type>> m_coroutine_return;
future_type m_current_dep;
```

Just like in the Fibonacci example above, we provide a call operator that Kokkos will invoke when all of the task's dependences are ready:

```
void operator()(typename Scheduler::member_type const& member, value_ty
```

We create an instance of the coroutine scheduler (the "wrapper" that we're building) to pass to the user's coroutine functor:


```
auto coro_scheduler = coroutine_scheduler_type{member.scheduler()};
```

Keeping in mind that we are going to respawn this functor, we need to check which respawn we're on in the body of the call operator. If the coroutine return object hasn't been created yet, we're on our first time through, and we should create it:

```
if(not m_coroutine_return) {  
    // initial invocation  
    m_coroutine_return = m_functor(coro_scheduler);  
}
```

Any time Kokkos calls in to this functor, all of the prerequisites of the task it represents will be ready. In this case, this means that the dependence we suspended for (if any) is ready, so we should resume the coroutine. First, we store a (non-owning) pointer to our future that regulates suspension in the promise, so that `await_transform` (and the `await_suspend` of the awaitable it returns) can find it:

```
m_coroutine_return->handle.promise().m_current_dep = &m_current_dep;
```

Then we resume the coroutine:

```
m_coroutine_return->handle.resume();
```

Now if the coroutine is done, we need to communicate the return value back to Kokkos (which is done by assigning to a reference passed in as a parameter) and destroy the coroutine handle:

```
if(m_coroutine_return->is_done()) {  
    value = m_coroutine_return->get_result();  
    m_coroutine_return->handle.destroy();  
}
```

Otherwise, we need to respawn with the suspension dependence as our prerequisite (this doesn't actually respawn the task in place, but rather marks the task for respawning and handles the respawn when the task returns):

```

else {
    Kokkos::respawn(this, m_current_dep, Kokkos::TaskPriority::High);

```

Futures in Kokkos are reference counted and cannot be reused once they're made ready, so we need to replace the future held by `this` with an empty one:

```

    m_current_dep = future_type{};
}

```

And that's it for the task functor:

```

}

};

```

We now need to implement the type returned by `scheduler.spawn()`, which stores the value returned by `Kokkos::task_spawn()` in a data member:

```

template <class ValueType>
struct SpawnedAwaitable {
    using value_type = ValueType;
    using scheduler_type = Scheduler;
    using value_future_type = Kokkos::BasicFuture<value_type, scheduler_type>;

    value_future_type m_done_future;

```

We provide an accessor for the user to interface with existing Kokkos tasking code:

```

value_future_type get_future() { return m_done_future; }

```

And then we need to implement the nested promise type, which is returned by `await_transform` when the user applies operator `co_await`. It stores a copy of the future from the `SpawnedAwaitable` (futures in Kokkos have reference semantics with shared ownership) and a reference to the `promise_type` instance from the enclosing coroutine:

```
template <class ParentPromise>
struct SpawnedPromise {

    value_future_type m_done_future;
    ParentPromise& m_parent_promise;
};
```

The `await_ready` hook simply checks if the future is ready:

```
bool await_ready() const {
    return m_done_future.is_ready();
}
```

And the `await_suspend` hook assigns the future to the one pointed to by the parent promise, which should be set to the `TaskFunc`'s future that controls suspension. (Note that non-`void` futures can be assigned to `void` futures in Kokkos.)

```
void
await_suspend(std::experimental::coroutine_handle<ParentPromise> handle) {
    *m_parent_promise.m_current_dep = m_done_future;
}
```

Finally, since the `TaskFunc` ensures that `resume()` is only called on the coroutine handle when the future is ready, `await_resume()` is trivial:

```
value_type&
await_resume() {
    return m_done_future.get();
}

};
```

The implementation of `WhenAllAwaitable` is similar, albeit messier:

```

template <class... Awaitables>
struct WhenAllAwaitable {
    using value_type = std::tuple<typename Awaitables::value_type...>;
    using scheduler_type = Scheduler;
    using aggregate_future_type = Kokkos::BasicFuture<void, scheduler_type>;
    using value_future_tuple = std::tuple<
        Kokkos::BasicFuture<typename Awaitables::value_type, scheduler_type>,
    >;

    aggregate_future_type m_done_future;
    value_future_tuple m_value_futures;

template <class ParentPromise>
struct SpawnedPromise {

    aggregate_future_type m_done_future;
    value_future_tuple m_value_futures;
    ParentPromise& m_parent_promise;

    SpawnedPromise(
        aggregate_future_type arg_done_future,
        value_future_tuple arg_value_futures,
        ParentPromise& arg_parent_promise
    ) : m_done_future(std::move(arg_done_future)),
        m_value_futures(std::move(arg_value_futures)),
        m_parent_promise(arg_parent_promise)
    { }

    bool await_ready() const {
        return m_done_future.is_ready();
    }

    void
    await_suspend(std::experimental::coroutine_handle<ParentPromise> hanc
        *m_parent_promise.m_current_dep = m_done_future;
    }

template <size_t... Idxs>
value_type
_await_resume_impl(
    std::integer_sequence<size_t, Idxs...>
)
{

```

```

        return std::make_tuple(
            (std::get<Idxs>(m_value_futures).get())...
        );
    }

    value_type
    await_resume() {
        return _await_resume_impl(std::index_sequence_for<Awaitables...>{})
    }
};
};

```

Finally, the user-facing `spawn()` and `when_all()` methods merely delegate to their corresponding implementations in Kokkos:

```

template <class CoroutineFunctor>
SpawnedAwaitable<typename CoroutineFunctor::value_type>
spawn(CoroutineFunctor functor, Kokkos::TaskPriority priority = Kokkos::1
    return {
        Kokkos::task_spawn(
            Kokkos::TaskSingle(m_scheduler, priority), TaskFunctor<CoroutineFunctor>
        )
    };
}

template <class... Awaitables>
WhenAllAwaitable<std::decay_t<Awaitables>...>
when_all(Awaitables&&... awaitables) const {
    future_type all_void[] = { (awaitables.m_done_future)... };
    return {
        Kokkos::when_all(
            all_void, sizeof...(Awaitables)
        ),
        std::forward<Awaitables>(awaitables)...
    };
}
};

```

And that's it. Notice nothing internal to Kokkos needed to be touched to make this work. We feel this is evidence that the current form of the Coroutines TS nicely complements existing practice.

§ 4. Benchmarks

In our informal benchmarking of the Fibonacci example given above, we found that the coroutine-wrapped version was consistently a bit *faster* than the non-coroutine version—that is, the abstraction actually has *negative* overhead. This is attributed to the fact that `await_ready` can check for the completion of the (eagerly spawned) task that the awaitable depends on and skip suspension altogether.

§ 5. Appendix: Source code

```
#include <Kokkos_Core.hpp>

#include <impl/Kokkos_Timer.hpp>

#include <cstring>
#include <cstdlib>
#include <limits>
#include <optional>
#include <algorithm>
#include <experimental/coroutine>
#include <tuple>

//-----
// version has to do because there's no way to short circuit the respawn de
// on a ready future without coroutines
// #define DISABLE_CHECK_IN_AWAIT_READY 1

//-----

struct BasicCoroutineScheduler {

    using future_type = Kokkos::BasicFuture<void, Scheduler>;

    template <class CoroutineFunctor>
    struct SpawnedAwaitable;
    template <class...>
    struct WhenAllAwaitable;

    template <class T>
    struct coroutine_return_type {
        using value_type = T;
        // assume value_type is not void for now

        struct promise_type;

        using coroutine_handle = std::experimental::coroutine_handle<promise_ty

        struct promise_type {

            std::optional<value_type> result;
            future_type* m_current_dep = nullptr;
```

```

// promise creation must be followed by a suspend in order to set up
std::experimental::suspend_always initial_suspend() { return { }; }
// co_return must be followed by a suspend in order to use the result
std::experimental::suspend_always final_suspend() { return { }; }

coroutine_return_type<value_type>
get_return_object() {
    return { coroutine_handle::from_promise(*this) };
}

template <class Value>
void return_value(Value&& value) {
    result = std::move(value);
}

template <class ValueType>
typename SpawnedAwaitable<ValueType>::template SpawnedPromise<promise_type>
await_transform(SpawnedAwaitable<ValueType>& awaitable) {
    return {
        awaitable.m_done_future,
        *this
    };
}

template <class... Awaitables>
typename WhenAllAwaitable<Awaitables...>::template SpawnedPromise<promise_type>
await_transform(WhenAllAwaitable<Awaitables...>const & awaitable) {
    return {
        awaitable.m_done_future,
        awaitable.m_value_futures,
        *this
    };
}

void unhandled_exception() { std::abort(); }

};

bool is_done() {
    return bool(handle.promise().result);
}

```



```

value_type& get_result() {
    return *handle.promise().result;
}

coroutine_handle handle;

};

template <class CoroutineFunctor>
struct TaskFunctor {

    using value_type = typename CoroutineFunctor::value_type;
    using coroutine_scheduler_type = BasicCoroutineScheduler;

    CoroutineFunctor m_functor;
    std::optional<coroutine_return_type<value_type>> m_coroutine_return;
    future_type m_current_dep;

    void operator()(typename Scheduler::member_type const& member, value_ty

        auto coro_scheduler = coroutine_scheduler_type{member.scheduler()};

        if(not m_coroutine_return) {
            // initial invocation
            m_coroutine_return = m_functor(coro_scheduler);
        }

        // no dependency once we get here
        assert(m_current_dep.is_null() == true || m_current_dep.is_ready());

        assert(m_coroutine_return->handle.promise().m_current_dep == nullptr);
        assert(m_current_dep.is_null());

        // put a pointer to our dep in the promise so that any co_await calls
        // will know what dependence to set for the respawn
        m_coroutine_return->handle.promise().m_current_dep = &m_current_dep;

        // resume the coroutine
        m_coroutine_return->handle.resume();

        // Reset the promises parent dep

```

```

    m_coroutine_return->handle.promise().m_current_dep = nullptr;

    // Either it was done to begin with, or done after we resumed above:
    if(m_coroutine_return->is_done()) {
        // destroy the coroutine handle
        assert(m_current_dep.is_null());
        value = m_coroutine_return->get_result();
        m_coroutine_return->handle.destroy();
    }
    else {
        // Respawn dependent on whatever caused the resume to not reach the
        Kokkos::respawn(this, m_current_dep, Kokkos::TaskPriority::High);

        // Reset our dependence, since we've handled the respawn
        m_current_dep = future_type{};
    }

}

};

template <class ValueType>
struct SpawnedAwaitable {
    using value_type = ValueType;
    using scheduler_type = Scheduler;
    using value_future_type = Kokkos::BasicFuture<value_type, scheduler_type>;

    value_future_type m_done_future;

    value_future_type get_future() { return m_done_future; }

    template <class ParentPromise>
    struct SpawnedPromise {

        value_future_type m_done_future;
        ParentPromise& m_parent_promise;

        SpawnedPromise(
            value_future_type arg_done_future,
            ParentPromise& arg_parent_promise
        ) : m_done_future(arg_done_future),
            m_parent_promise(arg_parent_promise)
        { }
    };

```

```

        bool await_ready() const {
#ifdef DISABLE_CHECK_IN_AWAIT_READY
            return false;
#else
            return m_done_future.is_ready();
#endif
        }

        void
        await_suspend(std::experimental::coroutine_handle<ParentPromise> handle) {
            // for some value_type T, handle.promise() is of type coroutine_return_type
            // We now have something to resume when the future is ready
            assert(m_parent_promise.m_current_dep != nullptr);
            // Tell the parent that it needs to respawn with this as a dependent
            *m_parent_promise.m_current_dep = m_done_future;
        }

        value_type&
        await_resume() {
            return m_done_future.get();
        }

    };

};

template <class... Awaitables>
struct WhenAllAwaitable {
    using value_type = std::tuple<typename Awaitables::value_type...>;
    using scheduler_type = Scheduler;
    using aggregate_future_type = Kokkos::BasicFuture<void, scheduler_type>;
    using value_future_tuple = std::tuple<
        Kokkos::BasicFuture<typename Awaitables::value_type, scheduler_type>,
        ...>;

    aggregate_future_type m_done_future;
    value_future_tuple m_value_futures;

    WhenAllAwaitable(
        aggregate_future_type&& done_future,
        Awaitables const&... awaitables
    ) : m_done_future(std::move(done_future)),

```

```

        m_value_futures(awaitables.m_done_future...)
    { }

template <class ParentPromise>
struct SpawnedPromise {

    aggregate_future_type m_done_future;
    value_future_tuple m_value_futures;
    ParentPromise& m_parent_promise;

    SpawnedPromise(
        aggregate_future_type arg_done_future,
        value_future_tuple arg_value_futures,
        ParentPromise& arg_parent_promise
    ) : m_done_future(std::move(arg_done_future)),
        m_value_futures(std::move(arg_value_futures)),
        m_parent_promise(arg_parent_promise)
    { }

    bool await_ready() const {
#ifdef DISABLE_CHECK_IN_AWAIT_READY
        return false;
#else
        return m_done_future.is_ready();
#endif
    }

    void
    await_suspend(std::experimental::coroutine_handle<ParentPromise> handle) {
        // for some value_type T, handle.promise() is of type coroutine_return_type
        // We now have something to resume when the future is ready
        assert(m_parent_promise.m_current_dep != nullptr);
        // Tell the parent that it needs to respawn with this as a dependent
        *m_parent_promise.m_current_dep = m_done_future;
    }

    template <size_t... Idxs>
    value_type
    _await_resume_impl(
        std::integer_sequence<size_t, Idxs...>
    )
    {
        return std::make_tuple(

```

```

        (std::get<Idxs>(m_value_futures).get())...
    );
}

value_type
await_resume() {
    return _await_resume_impl(std::index_sequence_for<Awaitables...>{})
}

};

};

template <class CoroutineFunctor>
SpawnedAwaitable<typename CoroutineFunctor::value_type>
spawn(CoroutineFunctor functor, Kokkos::TaskPriority priority = Kokkos::1

    return {
        Kokkos::task_spawn(
            Kokkos::TaskSingle(m_scheduler, priority), TaskFunctor<CoroutineFun
        )
    };
}

template <class... Awaitables>
WhenAllAwaitable<std::decay_t<Awaitables>...>
when_all(Awaitables&&... awaitables) const {

    future_type all_void[] = { (awaitables.m_done_future)... };
    return {
        Kokkos::when_all(
            all_void, sizeof...(Awaitables)
        ),
        std::forward<Awaitables>(awaitables)...
    };
}

template <class CoroutineFunctor>
SpawnedAwaitable<CoroutineFunctor>
spawn_team(CoroutineFunctor functor, Kokkos::TaskPriority priority = Kokk

```

```

        return {
            m_scheduler,
            Kokkos::task_spawn(
                Kokkos::TaskTeam(m_scheduler, priority), TaskFunctor<CoroutineFunctor>
            )
        };

    }

    Scheduler m_scheduler;

};

//-----
template <class Scheduler>
struct TestFibCoroutine {

    using scheduler_type = Scheduler;
    using value_type = long;
    using coroutine_scheduler_type = BasicCoroutineScheduler<scheduler_type>;
    using coroutine_return_type = typename coroutine_scheduler_type::template

    value_type n;

    coroutine_return_type
    operator()(coroutine_scheduler_type& sched) {
        if(n < 2) {
            co_return n;
        }
        else {
            auto f_2 = sched.spawn(TestFibCoroutine{n-2}, Kokkos::TaskPriority::High);
            auto f_1 = sched.spawn(TestFibCoroutine{n-1});
            co_return co_await f_1 + co_await f_2;
        }
    }
};

//-----
template <class Scheduler>
struct TestWhenAllFibCoroutine {

    using scheduler_type = Scheduler;

```

```

using value_type = long;
using coroutine_scheduler_type = BasicCoroutineScheduler<scheduler_type>;
using coroutine_return_type = typename coroutine_scheduler_type::template

value_type n;

coroutine_return_type
operator()(coroutine_scheduler_type& sched) {
    if(n < 2) {
        co_return n;
    }
    else {
        auto f_2 = sched.spawn(TestWhenAllFibCoroutine{n-2}, Kokkos::TaskPriorityLow);
        auto f_1 = sched.spawn(TestWhenAllFibCoroutine{n-1});
        auto [v1, v2] = co_await sched.when_all(f_1, f_2);
        co_return v1 + v2;
    }
}

};

//-----
template <class Scheduler>
struct TestFib {

    using MemorySpace = typename Scheduler::memory_space;
    using MemberType = typename Scheduler::member_type;
    using FutureType = Kokkos::BasicFuture<long, Scheduler>;

    using value_type = long;

    FutureType dep[2];
    const value_type n;

    KOKKOS_INLINE_FUNCTION
    TestFib(const value_type arg_n)
        : dep{}, n(arg_n)
    { }

    KOKKOS_INLINE_FUNCTION
    void operator()( const MemberType & member, value_type & result ) noexcept
    {
        auto sched = member.scheduler();

```

```

    if (n < 2) {
        result = n;
    }
    else if (!dep[0].is_null() && !dep[1].is_null()) {
        result = dep[0].get() + dep[1].get();
    }
    else {
        // Spawn new children and respawn myself to sum their results.
        // Spawn lower value at higher priority as it has a shorter
        // path to completion.

        dep[1] = Kokkos::task_spawn(
            Kokkos::TaskSingle(sched, Kokkos::TaskPriority::High),
            TestFib(n - 2)
        );

        dep[0] = Kokkos::task_spawn(
            Kokkos::TaskSingle(sched),
            TestFib(n - 1)
        );

        auto fib_all = Kokkos::when_all(dep, 2);

        // High priority to retire this branch.
        Kokkos::respawn(this, fib_all, Kokkos::TaskPriority::High);
    }
};

//-----
Kokkos::initialize(argc,argv);

{
    static constexpr auto N = 30;
    static constexpr auto repeats = 3;

    using scheduler_type = Kokkos::NewTaskSchedulerMultiple<Kokkos::OpenMP>
    using coroutine_scheduler_type = BasicCoroutineScheduler<scheduler_type>
    using memory_space = scheduler_type::memory_space;

    static constexpr size_t MinBlockSize = 64;
    static constexpr size_t MemoryCapacity = (N+1) * (N+1) * 2000;
    static constexpr size_t MaxBlockSize = 1024;

```



```

static constexpr size_t SuperBlockSize = 4096;

scheduler_type scheduler(
    memory_space(), MemoryCapacity, MinBlockSize,
    std::min(size_t(MaxBlockSize), MemoryCapacity),
    std::min(size_t(SuperBlockSize), MemoryCapacity)
);
auto coroutine_scheduler = coroutine_scheduler_type{ scheduler };

std::cout << "Running benchmark Fib(n) for 0 <= n < " << N << std::endl;
std::cout << "-----" << std::endl;
for(int irepeat = 0; irepeat < repeats; ++irepeat) {
    std::cout << "Benchmarking repeat #" << (irepeat + 1) << " of " << re

    {
        Kokkos::Impl::Timer timer;
        for(int i = 0; i < N; ++i) {
            auto result = coroutine_scheduler.spawn(TestFibCoroutine<schedul
            Kokkos::wait(scheduler);
        }

        std::cout << " Simple coroutine version took " << timer.seconds()
    }

    {
        Kokkos::Impl::Timer timer;
        for(int i = 0; i < N; ++i) {
            auto result = coroutine_scheduler.spawn(TestWhenAllFibCoroutine<s
            Kokkos::wait(scheduler);
        }

        std::cout << " Coroutine version with when_all took " << timer.sec

    }

    {
        Kokkos::Impl::Timer timer;
        for(int i = 0; i < N; ++i) {
            auto result_future =
                Kokkos::host_spawn(Kokkos::TaskSingle(scheduler), TestFib<sched
            Kokkos::wait(scheduler);
        }

        std::cout << " Old version took " << timer.seconds() << std::endl;
    }
}

```

```
    }  
  
    }  
  
} // end scope to destroy scheduler before finalize  
  
Kokkos::finalize();  
  
}
```